fastmcp_agentic_phase1_...
Zip Archive

Please study my project carefully. More instructions to follow...

I'm opening the project now and mapping the structure, entry points, and major components so I'm ready for your next instructions. I'll focus on how the pieces fit together, not just the file names.

I've got the high-level layout. The core shape is a FastAPI host with an MCP client runtime, an orchestrator layer, optional skill routing, and observability hooks around the chat path.

Thought for 1m 27s

I've studied the project and have the main flow mapped.

What I've confirmed:

- The app is a **FastAPI host** in `backend/main.py` that builds settings, starts the **MCP client runtime**, loads optional **Skills**, creates the **LangChain LLM/orchestrator**, and exposes `POST /chat`.
- The core decision loop lives in `backend/agent/orchestrator.py`. It does:
  1. optional **skill selection**
  2. optional **skill prompt loading**
  3. **tool filtering** by allowed tools
  4. `bind_tools(...)` on the LLM
  5. repeated **LLM → tool calls → LLM** turns
  6. telemetry capture for LLM/tool latency and usage
- Session memory is in `backend/agent/sessions.py` with an in-memory store keyed by `session_id`.
- MCP connectivity is in `backend/mcp/client.py`, using `MultiServerMCPClient` and manually opening long-lived sessions for each configured server.
- Skills are lightweight and deterministic:
  - `math`

    - `quote_count`
    - `data_analyst`

    with keyword routing from `skills/registry.json`.
- Observability is already a major part of the design:
    - JSONL metrics
    - optional OTEL traces/metrics
    - optional MLflow config
    - benchmark and comparison scripts in `scripts/` and saved runs in `benchmarks/`

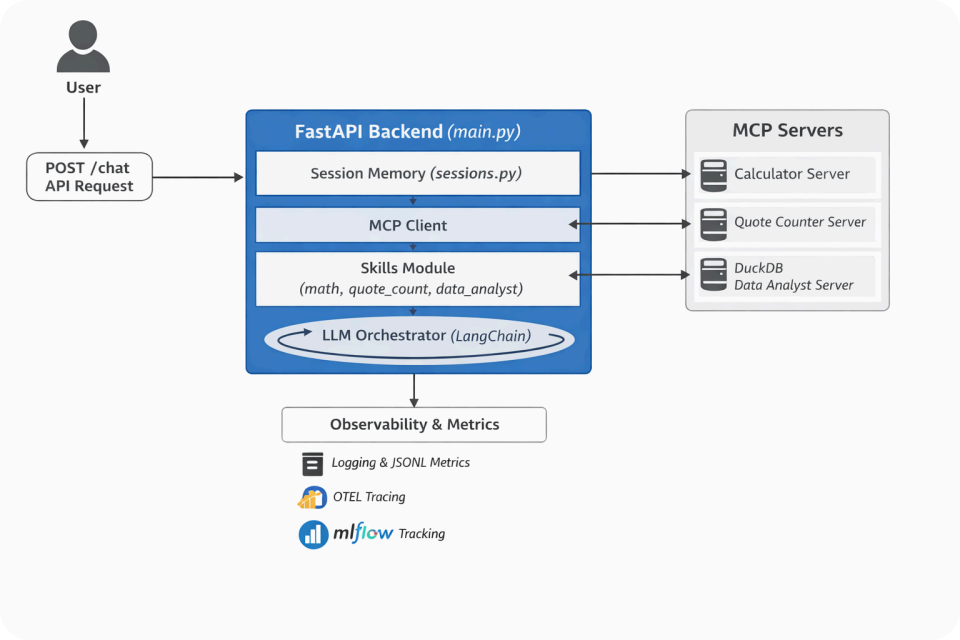
A few important things I noticed right away:

- `ChatSessionStore` uses **one global async lock** for all sessions, so concurrent users are effectively serialized.
- `.env.example` still includes older-looking `MCP_TRANSPORT` / `MCP_URL` settings, while the actual code path uses `MCP_SERVERS`.
- The project has grown beyond just calculator tooling; there are now **multiple MCP servers**, including a **DuckDB-backed data analyst server**.

I'm ready for the next instructions.

Okay so first can you give me a diagram of the architecture. Don't bloat the image too much with function calls, I'm mostly interested in the flow of the application and how all frameworks and services are used and when

Image created • **AI-driven application architecture flowchart**



can you explain the arrows to the MCP Servers, are the arrows 1-to-1 or are they merely pointing to that all call the MCP Server

Thought for 6s

They are **not meant to be strict 1-to-1 arrows**.

The better way to read that part of the diagram is:

- the **backend has access to a pool of MCP servers**
- the **MCP client layer manages the actual connections/sessions**
- the **orchestrator can use tools exposed by any connected server**
- the **skills module does not directly call a specific server**; it mainly helps decide which tools/server capabilities are relevant

So in practice, those arrows are really saying:

“these backend components rely on the MCP server layer”
not
“this box only talks to that one server”

A more accurate mental model is:

```
User -> FastAPI -> Orchestrator -> MCP Client -> one or more MCP Servers
```

And then:

- **Calculator requests** may end up using the calculator server
- **Quote count requests** may end up using the quote counter server
- **Data analysis requests** may end up using the DuckDB analyst server

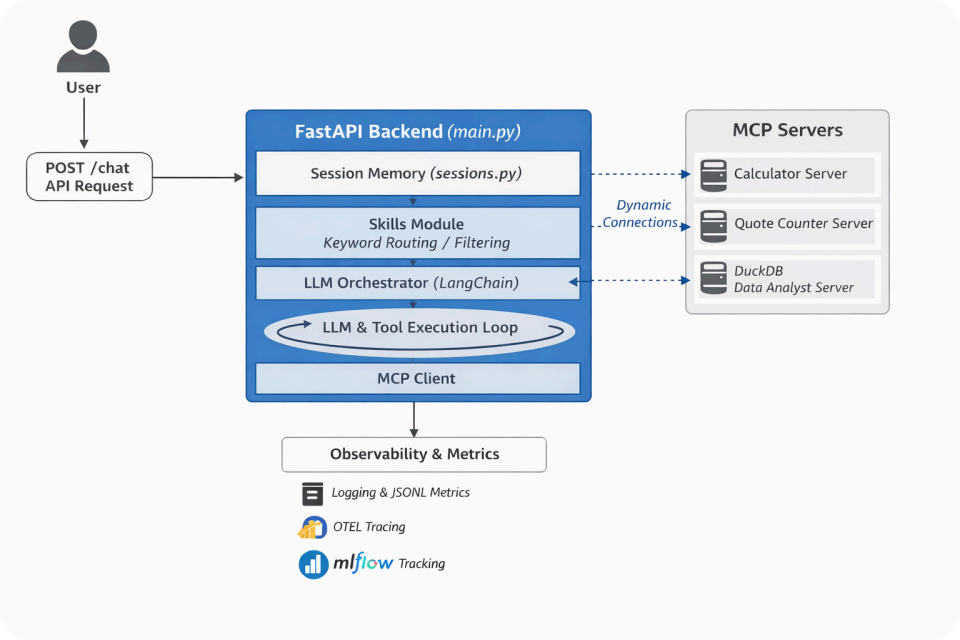
So the routing is **dynamic**, not hardwired 1:1.

One thing I'd correct from the image: the arrows from **Skills Module** to the individual MCP servers are a little misleading. Skills are better thought of as a **routing/filtering layer inside the backend**, while the **MCP client** is the thing that actually interfaces with the servers.

If you want, I can make you a cleaner revised architecture diagram with that distinction fixed.

yes please do

Image created • **FastAPI chatbot architecture overview**



Okay the reason I want to make sure we have the architecture correctly is because I have been given a task to implement deterministic workflows. From my own research, I think something like LangGraph could be a good fit. What do you think?

I'm checking two things together: whether LangGraph is a good fit in general, and whether it fits *your current architecture specifically* without forcing a rewrite. Then I'll give you a concrete recommendation, not just theory.

I've confirmed the key architectural detail that matters here: your current setup is only *partly* deterministic. Skill selection is deterministic, but once the model enters the tool loop, the next actions are still largely model-driven.

Thought for 1m 24s

Yes — for **your specific assignment of adding deterministic workflows**, LangGraph looks like a **good fit**.

The key reason is that LangGraph explicitly separates **workflows** from **agents**. In the official docs, workflows are described as having **predetermined code paths** and operating in a set order, while agents are **dynamic** and choose their own process and tool usage. LangGraph's model is built around **state, nodes, and edges**, including **conditional edges**, which is exactly the right mental model for "go from A → B → C, with controlled branching." LangChain Docs +1

It also fits your stack better than it might seem at first. LangGraph is a **low-level orchestration framework** rather than a high-level "magic agent" layer, so it does not force you to abandon your current prompts, tools, or architecture. Its docs are pretty clear that it is meant for **long-running, stateful workflows/agents** and gives you low-level control rather than abstracting away architecture decisions. LangChain Docs +1

For your project specifically, the biggest issue today is that it is only deterministic at the **front** of the request:

- skill selection is deterministic
- allowed-tool filtering is deterministic

But after that, your `Orchestrator.respond()` enters a model-driven tool loop:

- bind visible tools
- call the LLM
- inspect returned tool calls
- execute them
- feed results back
- repeat until the model stops

That means the **overall path is still agentic**, even though the entry routing is controlled.

So my real recommendation is:

Use LangGraph for deterministic workflows, but do not replace your whole app with LangGraph all at once.

The best architecture for you is probably **hybrid**:

1. Keep your current FastAPI host, MCP runtime, metrics, and session plumbing.
2. Keep your current freeform orchestrator for normal chat.
3. Add **specific LangGraph workflows** for tasks that must be deterministic.

That would give you something like:

- **Freeform chat path** → your current orchestrator
- **Deterministic workflow path** → LangGraph graph
- both paths can still use the same MCP-backed tools

That last part matters a lot: LangChain's MCP docs and LangChain's own announcement both say the MCP adapters make MCP tools usable with **LangChain and LangGraph**, including tools across **multiple MCP servers**. So your calculator server, quote counter server, and data analyst server can stay part of the design instead of being rewritten.

changelog.langc... +1

Where LangGraph would help you most in *this* codebase:

- **deterministic multi-step data workflows**
  Example: validate request → inspect schema → run SQL → validate result → summarize
- **approval / human-check workflows**
  Example: draft → review gate → revise → finalize

- **error recovery with explicit branches**
  Example: if SQL fails, route to repair node; if repair fails twice, return structured error
- **mode-specific execution**
  Example: route "data analyst" to a graph, while normal conversation still uses the current orchestrator

A good first implementation would be very small. I would not start with a giant graph. I'd start with **one workflow**, probably the data analyst path, because it is the most naturally procedural. In LangGraph terms, that could be:

- `route_request`
- `validate_inputs`
- `list_or_check_dataset`
- `inspect_schema`
- `generate_query`
- `execute_query`
- `repair_or_retry_if_needed`
- `summarize_result`
- `finish`

That is much easier to reason about than letting one LLM loop decide everything dynamically.

The main caution is this: LangGraph is **not** the right choice if what you really need is only a couple of simple `if/else` workflows. If you only need a tiny number of deterministic paths, a plain Python state machine or explicit service-layer router would be simpler. LangGraph starts to pay off when you want:

- branching workflows
- reusable state
- checkpoints/persistence
- mixed deterministic + agentic paths
- more workflows coming later

Given the direction of your project, I think you are already close to that threshold.

So my honest take is:

Yes, LangGraph is a strong fit for your deterministic-workflow task.
Not because "LangGraph is trendy," but because your current app already has:

- routing
- tools
- state
- observability
- multiple backends/services

What it lacks is an explicit **workflow engine** between routing and tool execution. LangGraph fits that gap well. LangChain Docs +3

My recommendation in one sentence:

Adopt LangGraph as a new deterministic-workflow layer inside your existing FastAPI/MCP app, not as a full replacement for the current orchestrator.

The next best step would be to map your current architecture into:

- what stays as-is
- what becomes a LangGraph node
- what remains normal agentic chat

Sources

yes please do. Give me a clear picture of where LangGraph would plug in with the architecture diagram you drew for me

Thought for 17s

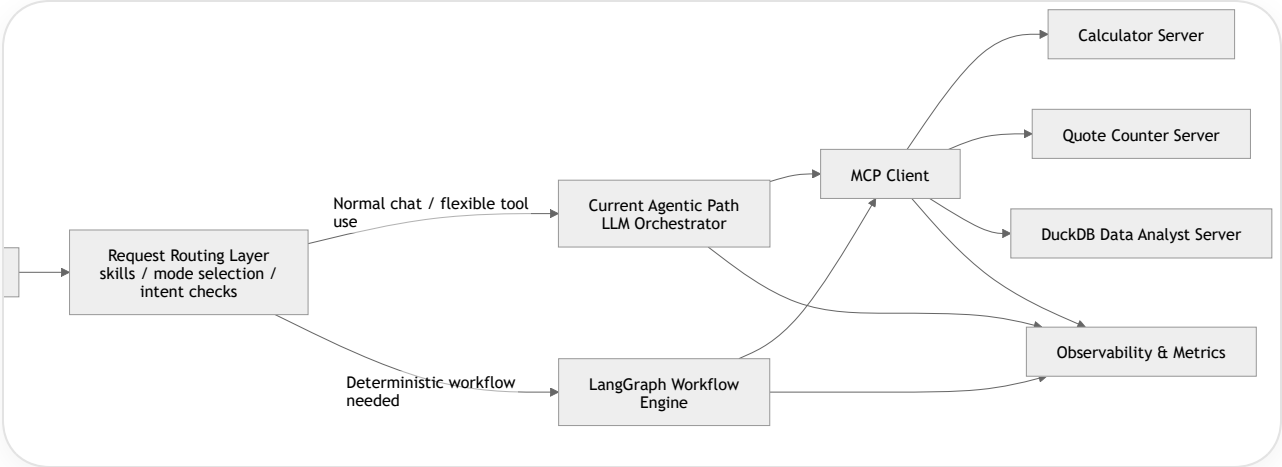
Yes — the cleanest way to think about it is that **LangGraph plugs in between request routing and tool execution**.

Your app today is roughly:

```
User
  ->
FastAPI /chat
  ->
Session Memory
  ->
Skills Routing / Tool Filtering
```

```
  ->
LLM Orchestrator
  ->
MCP Client
  ->
MCP Servers
```

With LangGraph added, the architecture becomes **two execution paths** inside the backend:



That means LangGraph does **not** replace FastAPI, Sessions, MCP, or Observability.

It becomes a new orchestration layer for the cases where you want:

- fixed steps
- explicit branches
- retries
- approval gates
- predictable A → B → C execution

The most important architectural point

Right now your **LLM Orchestrator** is doing two jobs at once:

1. deciding what kind of request this is
2. running the execution loop

LangGraph should take over job **2** for deterministic cases.

So the new mental model is:

- **Skills / router** decide whether this request is:
  - freeform chat
  - deterministic workflow

- then:
  - freeform chat goes to your current orchestrator
  - deterministic workflow goes to LangGraph

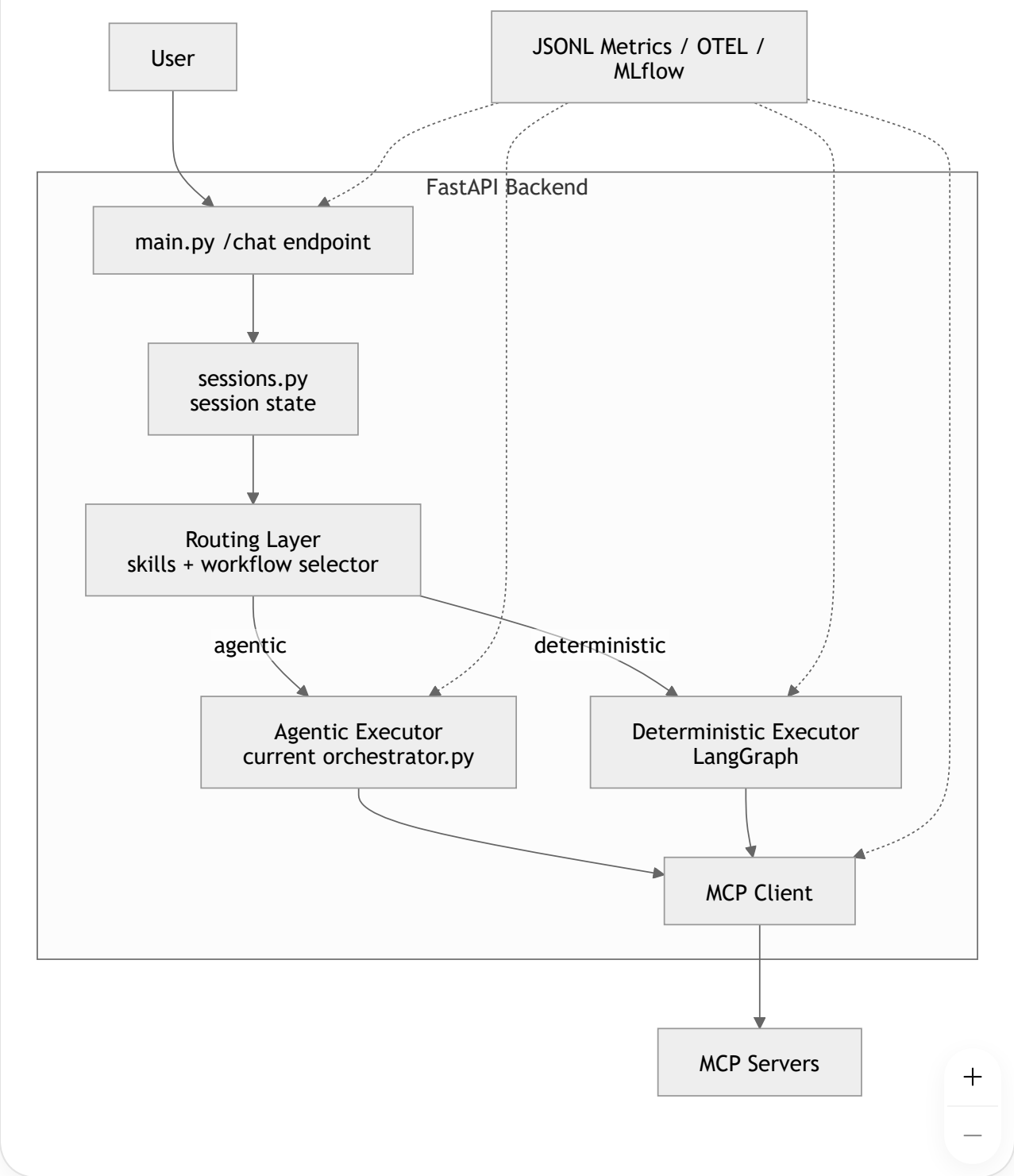
Where it plugs in, in plain English

Inside your backend, the flow would become:

```
POST /chat
  ->
load session
  ->
route request
  ->
if workflow is deterministic:
    run LangGraph graph
else:
    run current LLM orchestrator
  ->
return response
```

The clean architecture version

I'd draw your backend like this:



What stays the same

These parts likely remain as-is:

- `backend/main.py`
- `backend/mcp/client.py`
- `backend/agent/sessions.py`
- your MCP servers
- your metrics / OTEL / MLflow setup

- likely your skills registry, though its role becomes more important

What changes

You would add a new layer, something like:

```
backend/workflows/
    base.py
    registry.py
    data_analyst_graph.py
    approval_graph.py
    simple_sequential_graph.py
```

And then the router decides:

- use `orchestrator.respond(...)`
- or use `workflow_runner.run(...)`

Where LangGraph is most natural in your project

The **best first fit** is probably the `data_analyst` path.

Instead of:

- let the LLM decide all next steps dynamically

you make it:

```
identify data request
  ->
inspect available tables/schema
  ->
build query
  ->
execute query
  ->
if query fails, repair once or twice
  ->
summarize result
  ->
return
```

That is exactly the kind of thing LangGraph is good at.

The key benefit for your architecture

Today, the orchestrator is a **single intelligent loop**.

With LangGraph, your backend becomes a **dispatcher**:

- **agentic path** for open-ended chat
- **graph path** for deterministic workflows

That is the clean picture of where it plugs in.

My recommendation for your codebase

I would structure it conceptually like this:

```
FastAPI Backend
├── Session Memory
├── Routing Layer
│   ├── Skill selection
│   ├── Workflow selection
│   └── Tool visibility rules
├── Execution Engines
│   ├── Current LLM Orchestrator
│   └── LangGraph Workflow Runner
├── MCP Client
└── Observability
```

So the answer in one sentence is:

LangGraph should plug in as a second execution engine inside the backend, after routing/skill selection and before MCP tool execution.

If you want, I can make you a revised visual diagram image that shows the **current architecture vs. architecture with LangGraph added** side by side.